FURTHER STUDY

.NET Tech Lead

Next 20 days

Azure Pipelines · TFS and Azure DevOps · GitHub Actions

ASP.NET Core MVC · MVVM · Microservices · AI-assisted delivery

This follows the 10-day interview roadmap and the deep dive. Those days taught the API, SQL, Azure hosts, and a first pass at microservices. These 20 days teach how the change reaches production, how MVC and MVVM differ, and how to use Cursor and GitHub Copilot without skipping review.

Prepared 31 October 2026. Examples stay on .NET 10, ASP.NET Core, Azure App Service, and Azure Pipelines or GitHub Actions. Do not memorize a price or a quota. Name the document you would check.

How the next 20 days fit

Each day is about 3.5 to 4 hours. Read the explanation, type the example, then say the short answer out loud. Days 1 to 10 are delivery. Days 11 to 16 are the UI patterns and a deeper ASP.NET Core pass. Days 17 and 18 extend microservices past the interview sketch. Days 19 and 20 are the AI-assisted product development life cycle, called AIPDLC here: where a model may draft, and where a human still signs.

Days	You will be able to teach
1-3	Git on a team, GitHub pull requests, and a GitHub Actions build for the orders API.
4-6	What TFS means now, an Azure Pipelines YAML file, and a CI gate that fails a bad pull request.
7-10	Deploy to a slot, identity without a password in the pipeline, and a rollback you have named.
11-14	ASP.NET Core MVC, and MVVM as a different shape used by a client, not a rename of MVC.
15-16	Middleware, filters, caching, background work, and an integration test.
17-18	A service boundary you can deploy on its own, with its own pipeline and data.
19-20	Where AI sits in the lifecycle, and the Cursor and Copilot habits that survive review.

Names you will hear in the same conversation

TFS is the old name. The on-premises product is Azure DevOps Server. The cloud product is Azure DevOps Services. People still say TFS when they mean either. GitHub is a different host. Azure Pipelines is the CI and CD engine inside Azure DevOps. GitHub Actions is the CI and CD engine inside GitHub. A team picks one engine for a service. Running both without a reason is two places to look during an incident.

Day 1 — Git the way a team uses it

Outcome: you can explain a branch, a pull request, a merge, and a rebase with the orders API as the example, and you can repair a conflict without rewriting history that other people already pulled.

Time box (3.5 hours)

- 40 min: commit, branch, and remote. Say each in one sentence.
- 80 min: type the example flow on a scratch repo.
- 50 min: merge versus rebase, then a conflict.
- 40 min: speak the Say this boxes.

The objects that matter

A commit is a snapshot plus a pointer to its parent. A branch is a movable name for a commit. HEAD is the commit you have checked out. origin is the usual name of the remote. main is the branch you protect. A feature branch is where the orders change lives until review. A pull request is the conversation and the policy gate. It is not a Git object. GitHub and Azure DevOps add it on top of the commits.

```
git switch -c feature/order-idempotency
# edit CreateOrder, add the test
git add src/Orders.Api tests/Orders.Tests
git commit -m "Store the idempotency key with the order"
git push -u origin feature/order-idempotency
# open the pull request into main
```

The commit message says why. The diff says what. A message such as 'updates' fails review because the history becomes useless at midnight. One commit can be several files when they are one decision. Two unrelated fixes are two commits, or two pull requests.

Merge and rebase

Merge joins two lines and can create a merge commit with two parents. History keeps the branch shape. Rebase replays your commits on top of a new base, so the feature looks like it started from today's main. Rebase rewrites commit ids. That is safe on a branch only you use. It is hostile on a branch other people have pulled, because their next pull must reconcile two histories of the same work.

```
git switch feature/order-idempotency
git fetch origin
git rebase origin/main
# conflict in CreateOrder.cs
# edit the file, keep both the new token check and your key storage
git add src/Orders.Api/CreateOrder.cs
git rebase --continue
git push --force-with-lease
```

force-with-lease refuses the push if someone else added a commit you have not seen. A plain force push can delete that commit. After the rebase, the pull request updates to the new commits. Squash merge on the server turns the feature into one commit on main. That is a team rule. Pick squash or merge and stay with it. Mixing them makes git bisect harder.

What breaks

Rebasing main, or rebasing a shared release branch, rewrites the commits the pipeline and other developers already built. Committing secrets, even in one commit you later delete, leaves them in the object database until history is rewritten and every clone is replaced. A conflict resolved by taking 'ours' without reading both sides drops the idempotency check.

Say this

A branch is a name for a commit. A pull request is the review gate. I rebase only the branch I own, and I push it with force-with-lease. I do not rewrite main.

Evening drill

1. Draw main, your branch, and the pull request. Mark which commits the build runs.
2. Explain why force-with-lease is the push you use after a rebase.
3. Name one file that must never be committed: appsettings with a password, a .pfx, a user secret file.

Day 2 — GitHub for the orders API

Outcome: you can set the repository rules a two-year developer should expect, and you can review a pull request for the contract, not the brace style.

Time box (3.5 hours)

- 30 min: repository, fork, and clone.
- 70 min: branch protection and CODEOWNERS.
- 70 min: a pull request review of a bad orders change.
- 40 min: secrets, environments, and what does not belong in the repo.

What GitHub adds on top of Git

GitHub hosts the remote, the pull request, the review comments, the checks, and the permissions. A fork is a separate repository used when you do not have write access. Inside a company, developers usually push branches to the same repository and open a pull request. Collaborators get a role: read, write, or admin. Admin is rare. Write does not mean permission to push to main if branch protection is on.

```
# .github/CODEOWNERS
/src/Orders.Api/Auth/        @contoso/api-security
/src/Orders.Api/**/*.cs      @contoso/orders-api
/.github/workflows/          @contoso/platform
/infra/                      @contoso/platform
```

CODEOWNERS names who must review a path. Auth and workflow files should not be approvable by the author alone. Branch protection on main requires a pull request, requires the CI check to pass, and dismisses a stale approval when new commits arrive. A ruleset is the newer GitHub form of the same idea. The lead answer is the rule, not the screen name.

A review that catches the bug

The diff adds a client-supplied total and logs the bearer token. The build is green. Green means it compiled and the existing tests passed. It does not mean the change is safe.

```
[HttpPost("/orders")]
public async Task<OrderDto> Create(OrderDto input, CancellationToken ct)
{
    logger.LogInformation("Token {Token}", Request.Headers.Authorization);
    var order = new Order { Total = input.Total, Sku = input.Sku };
    await db.SaveChangesAsync(ct);
    return input;
}
```

- The token in the log is a secret in production logs. Delete that line.

- Total from the body is the client setting the price. The server prices the order.
- There is no idempotency key, so a retry creates a second order.
- The method returns the client object, not the stored order.
- Ask for a test that a second POST with the same key does not insert again.

What breaks

Approving because the author is senior. A required check that is not the test job, so a skipped workflow still looks green. A secret committed in appsettings.Production.json. GitHub secret scanning may catch it later. The pipeline already cloned it.

Say this

Main requires a pull request and a passing build. CODEOWNERS covers auth, workflows, and infrastructure. I review the contract and the secrets, not the indentation.

Secrets and environments

A GitHub Actions secret is encrypted and masked in logs. An environment such as production can require a reviewer before a job that names that environment runs. Variables are for values that are not secret, such as an Azure client id if your org treats it as public. Subscription and tenant ids are often stored as secrets so logs stay masked. The application secret, the SQL password, does not live in the workflow file. Production should not have a SQL password at all. The app uses a managed identity. The workflow uses OpenID Connect, which Day 9 covers.

Say this

Secrets stay in the host's secret store. Environments gate production. I do not print a token to prove the header arrived.

Evening drill

1. List the three paths you would put in CODEOWNERS for the orders system.
2. Review the bad Create method out loud in under two minutes.
3. Say where a production SQL password must not be stored.

Day 3 — GitHub Actions for a .NET API

Outcome: you can read a workflow and point at the trigger, the permissions, the job, and the step that would deploy. You can explain why a pull request workflow does not deploy to production.

The shape of a workflow

A workflow file lives in .github/workflows. on says when it runs. permissions says what the GITHUB_TOKEN may do. jobs run in parallel unless one needs another. steps run in order on a runner. uses pulls an action. run executes a shell command. A pull request runs the build and the tests. A push to main may deploy, and only through an environment that has a reviewer.

```
name: orders-ci
on:
  pull_request:
  push:
    branches: [main]

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: "10.0.x"
      - run: dotnet restore Orders.sln
      - run: dotnet build Orders.sln -c Release --no-restore
      - run: dotnet test Orders.sln -c Release --no-build
```

Pin third-party actions. A moving tag such as @v4 can still change. A commit SHA is stricter. Use the major your security team has approved if they have not moved to SHAs yet, and say that. timeout-minutes stops a hung test from holding a runner. contents: read is enough for a test job. A job that pushes a package or comments on a pull request needs a narrower extra permission, not write on everything.

Cache, path filters, and artifacts

setup-dotnet can cache NuGet. A path filter stops a docs-only change from deploying the API. An artifact is the published output you pass to a later job. Build once. Deploy that output. Do not build again on the production runner, or the bits can differ.

```
on:
  push:
    branches: [main]
```

```
    paths:
      - "src/Orders.Api/**"
      - "tests/Orders.Tests/**"
      - ".github/workflows/orders-ci.yml"

jobs:
  publish:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: "10.0.x"
      - run: dotnet publish src/Orders.Api -c Release -o "$RUNNER_TEMP/api"
      - uses: actions/upload-artifact@v4
        with:
          name: orders-api
          path: ${{ runner.temp }}/api
```

What breaks

A workflow with on: pull_request and a deploy step will try to deploy every fork's code if you are not careful. Deploy jobs belong on a protected environment, on main, or on a manual workflow_dispatch. pull_request_target runs in the base repository and can see secrets. Do not check out the pull request head and run its scripts in that context. That is a secret leak.

Say this

Pull requests build and test. Main deploys the artifact those tests already built. Permissions start at read. I do not give a pull request workflow the production secret.

Evening drill

1. Point at trigger, permissions, job, and step in the workflow above.
2. Say why the deploy is not in the pull request job.
3. Name the risk of pull_request_target plus a checkout of the untrusted head.

Day 4 — TFS, Azure DevOps, and the words people mix

Outcome: you can translate TFS language into the current product, and you can say whether the team is on Git or TFVC, YAML or classic releases, Azure Repos or GitHub.

The family tree

Team Foundation Server was the on-premises server: work items, TFVC or Git, and XAML or classic builds. Microsoft renamed the server product to Azure DevOps Server. Azure DevOps Services is the same ideas, hosted by Microsoft, at dev.azure.com. A project contains boards, repos, pipelines, test plans, and artifacts. An organization contains projects. GitHub is not inside that box. A company can use GitHub for code and Azure DevOps for boards, or Azure Repos for code and Azure Pipelines for the build. Ask which combination they run before you design a second one.

They say	They usually mean	What you prefer now
TFS	Azure DevOps Server, or old habits	Name the version. Do not assume the cloud.
TFVC	Centralized version control, check-in	Git, unless a tool still requires TFVC.
Gated check-in	Build must pass before the check-in lands	A pull request policy with a required build.
Build definition	Classic visual build	A YAML file in the repo.
Release definition	Classic stages across environments	YAML stages and an Azure DevOps environment with approvals.
VSTS	The old cloud name	Azure DevOps Services.

Boards and the work the pipeline deploys

A work item is a user story, a bug, or a task. The pull request links to the work item so the board shows what shipped. That link is not a test. A story can move to Done while the slot swap failed. The lead habit is: the pipeline stage that deploys production is the moment the work is done, and the board follows that, or you will report features the customer cannot call.

```
# Pull request description
AB#1842 Idempotent POST /orders

The unique key is (TenantId, IdempotencyKey).
Rollback: slot swap, schema change is a nullable column only.
```

AB#1842 is the Azure Boards link form many teams use. GitHub issues use a different link. The content that matters is the same: the work id, the decision, and how to undo it.

TFVC versus Git, in one minute

TFVC has one server history. A check-in is immediate on the server path. Branches are server folders and merges are heavier. Gated check-in shelves the changes and builds them before they become a changeset. Git clones the history. The pull request is the gate. New .NET work goes to Git. If the interview is about a TFS estate that is still TFVC, the migration plan is the same shape as the Framework strangler: one team, one repo, a date, and a freeze on new TFVC branches. You do not convert 80 repositories in a weekend.

What breaks

Calling Azure DevOps 'GitHub' in a design review and then writing a workflow file the server cannot run. A classic release that only one person can edit, with no file in the repo, so the deploy steps are invisible in the pull request that changed the app. TFVC cloaking and workspace mappings that omit the new test project, so CI is green because it never compiled the tests.

Say this

TFS is the old name for Azure DevOps Server. I ask whether code is Git or TFVC and whether the pipeline is YAML or a classic release. New work is Git and YAML, reviewed like application code.

Evening drill

1. Translate gated check-in, build definition, and release definition into Git words.
2. Say which host you would pick if the company already pays for GitHub Enterprise, and what you would not rebuild.
3. Write the three lines you want on every pull request: work id, decision, rollback.

Day 5 — Azure Pipelines YAML

Outcome: you can walk a multi-stage pipeline from pull request to a staging slot, and you can say which parts are not allowed to run on a fork.

Stages, jobs, and steps

A pipeline is the file, usually azure-pipelines.yml. A stage is a boundary you can approve, such as Build and then DeployStaging. A job is a unit of work on an agent. Steps are tasks or scripts inside the job. Jobs in a stage run in parallel unless you set dependsOn. Stages run in order when you set dependsOn. A deployment job is the kind that records a deployment to an environment. An ordinary job does not.

```
trigger:
  branches:
    include: [main]
pr:
  branches:
    include: [main]

pool:
  vmImage: ubuntu-latest

variables:
  buildConfiguration: Release
  dotnetVersion: "10.x"

stages:
  - stage: Build
    jobs:
      - job: BuildTest
        steps:
          - task: UseDotNet@2
            inputs:
              packageType: sdk
              version: $(dotnetVersion)
          - script: dotnet restore Orders.sln
          - script: dotnet build Orders.sln -c $(buildConfiguration) --no-restore
          - script: dotnet test Orders.sln -c $(buildConfiguration) --no-build
          - script: dotnet publish src/Orders.Api -c $(buildConfiguration) -o
$(Build.ArtifactStagingDirectory)/api
          - publish: $(Build.ArtifactStagingDirectory)/api
            artifact: orders-api

  - stage: DeployStaging
    dependsOn: Build
    condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
    jobs:
      - deployment: Slot
        environment: orders-staging
        strategy:
          runOnce:
```

```
            deploy:
              steps:
                - download: current
                  artifact: orders-api
```

The pull request runs the Build stage because of the pr trigger. DeployStaging's condition requires main, so a pull request does not deploy. The environment orders-staging is where you attach an approval check if staging is shared. Production gets a stricter environment. The YAML in the repo is the review surface. A classic release edited in the browser is not in the pull request diff.

Variables, templates, and who can change them

A variable in the YAML is visible to anyone who can read the file. A secret variable or a variable group marked secret is masked. A variable group linked to Key Vault reads the secret at run time. Templates keep three services from copying the same build steps. A template is code. Review it like code. A parameter is the input. A variable is the value. Do not put the production connection string in a parameter default.

```
# template: pipelines/dotnet-test.yml
parameters:
  solution: Orders.sln
steps:
  - script: dotnet test ${{ parameters.solution }} -c Release

# caller
steps:
  - template: pipelines/dotnet-test.yml
    parameters:
      solution: Orders.sln
```

What breaks

A condition that checks Build.SourceBranchName against main while the real value is refs/heads/main. The deploy never runs, or a looser check lets a pull request deploy. A template parameter inserted into a script without care becomes command injection if the value comes from a pull request title. Checkout of a pull request plus a secret variable on the same job publishes the secret if the PR can change the script.

Say this

Build on every pull request. Deploy only from main, as a deployment job, into an environment. The YAML is in the repo. Secrets are in a variable group or Key Vault, not in the file.

Evening drill

1. Mark the stage a pull request is allowed to run.
2. Say what an environment approval is for.
3. Explain why you publish the API in Build and download it in Deploy.

Day 6 — The CI gate

Outcome: a pull request cannot merge when the tests fail, the solution does not build, or a migration was not reviewed. You can name the checks and the ones you will not pretend are security.

What the gate actually proves

Restore proves the packages resolve. Build proves the code compiles, including nullable warnings if you treat them as errors. Test proves the checks you wrote. It does not prove the ones you forgot. A formatting check proves whitespace. A dependency scan proves known vulnerable package versions, not a logic bug. A required reviewer on the migration folder proves a human saw the SQL. Stack those. Do not call the unit test job a security review.

```
dotnet test Orders.sln -c Release --no-build ^
  --collect:"XPlat Code Coverage" ^
  --logger "trx;LogFileName=results.trx"

# Treat warnings as errors on the API project once the backlog is clear.
# A new warning fails the pull request. An old warning is a ticket, not a silent ignore.
```

Coverage is a conversation, not a trophy. A lead asks which money path is untested, not whether the number is 80. The idempotency test and the tenant isolation test are required. A test that only asserts the controller returns 200 with a fake that never touches the rule is not. Integration tests that need SQL run in CI against a container or a dedicated database, not against production.

Pull request policies

On Azure DevOps, a branch policy on main requires a build validation, a minimum number of reviewers, and optionally a comment resolution. On GitHub, a ruleset does the same with a required check. The build validation uses the YAML. The policy lives in the project settings, which is a gap: the YAML is reviewed in Git, the policy is clicked in the portal. Write the policy down in the repo README or a pipeline doc so a new project copies it. Reset votes when the source changes, or an approval survives a later commit that adds the token log.

What breaks

A pipeline that uses continueOnError on the test step. The check is green and the tests failed. A path filter that excludes the test project, so production code changes and the tests do not run. Commenting out a failing test to merge. That is a lead conversation, not a style nit.

Say this

The required check builds and tests. Reviewers cover auth, schema, and workflows. Coverage points me at the untested money path. A green build with continueOnError is not a gate.

Evening drill

1. List four checks you would require on main, and one you would not call security.
2. Say what you do when a test fails and the release is today.
3. Find continueOnError in a sample and say what the board will show.

Day 7 — Continuous delivery to App Service

Outcome: you can deploy the artifact from Day 5 to a staging slot, smoke-test it, swap, and swap back. The schema change in the same release is still expand-only.

Build once, promote the artifact

The build stage publishes the API. Staging downloads that artifact and deploys it to the staging slot. Production swaps, or deploys the same artifact to the production slot after an approval. Nobody rebuilds between staging and production. A rebuild can pick up a newer package or a different commit if the trigger moved. The artifact name and the build id are what you write on the release ticket.

```
- task: AzureWebApp@1
  displayName: Deploy to staging slot
  inputs:
    azureSubscription: sc-orders-nonprod
    appType: webApp
    appName: app-orders
    deployToSlotOrASE: true
    resourceGroupName: rg-orders
    slotName: staging
    package: $(Pipeline.Workspace)/orders-api/**/*.zip
```

The task name and inputs drift by major version. The decision does not. The service connection is the Azure identity of the pipeline. The slot is staging. The package is the artifact, not a fresh publish. After deploy, hit /health/ready on the slot hostname. Then swap. Sticky settings stay with the slot. A staging connection string marked sticky by mistake means the smoke test hit the wrong database, or production starts using the staging database after the swap.

Approvals and the order of operations

The order is: migration as an expand, deploy to the slot, readiness, smoke, approval if a human must look, swap, watch the chart, leave the old bits on the other slot. The migration identity is not the App Service identity. The pipeline identity that deploys the zip is not automatically the identity that should run DDL. Split them. A manual approval on the production environment is how you stop a Friday deploy when the check is optional. It is not a substitute for the test gate.

What breaks

Swap before the migration, so the new bits read a column that does not exist. Swap after a destructive migration, so swap-back is a lie. Deploying the zip to production with no slot because slots felt like extra work, and then having no previous bits to return to. A smoke test that only checks that the process started, and never calls a real authenticated route.

Say this

One artifact, staging slot, readiness, then swap. Rollback is swap-back, and it is real only when the previous bits understand the current schema. Approvals sit on the environment. They do not replace tests.

Evening drill

1. Write the six steps from merge to swap, in order.
2. Say which identity runs DDL and which identity the site uses.
3. Name a smoke check that would catch a missing auth policy.

Day 8 — Functions, containers, and a small Bicep change

Outcome: the email function and the API do not share one pipeline stage by accident. You can show a small infrastructure change in source control.

A second artifact

The API zip and the function zip are different publish outputs. They can be two jobs in one pipeline or two pipelines with path filters. Path filters are how a README change avoids a production deploy, and how an email-only change avoids recycling the API. Each deploy uses its own identity. The function app's identity can send mail and complete Service Bus messages. It cannot alter the orders schema.

```
- script: dotnet publish src/Orders.Email -c Release -o $(Build.ArtifactStagingDirectory)/email
- publish: $(Build.ArtifactStagingDirectory)/email
  artifact: orders-email

# Deploy with the function app task your org standardizes on.
# The decision: a different app, a different identity, the same build id.
```

Containers when the host is not a zip

If the platform standard is Container Apps or App Service for containers, the artifact is an image, not a zip. The pipeline builds the image, pushes it to Azure Container Registry, and deploys the digest. A tag such as latest is not a version. The digest is. The same rule holds: build once, deploy that digest to staging, then to production. A Dockerfile that runs as root and copies secrets into a layer fails review even when the app starts.

Bicep for the change you can review

Clicking a slot in the portal does not show up in the pull request. A small Bicep file does. You do not need a full landing zone on this day. You need one resource the team can diff: the app, the staging slot, and the identity. What-if, or a plan, runs before the apply. Production apply sits on the production environment approval. Application code and infrastructure can be the same pipeline or a separate one. Separate is clearer when infrastructure changes are rare and dangerous. The application pipeline should not be able to delete the resource group.

```
resource app 'Microsoft.Web/sites@2023-12-01' = {
  name: 'app-orders'
  location: resourceGroup().location
  identity: {
    type: 'SystemAssigned'
  }
  properties: {
    serverFarmId: plan.id
    httpsOnly: true
    siteConfig: {
```

```
      alwaysOn: true
      ftpsState: 'Disabled'
    }
  }
}
```

The API version on the resource type changes. The properties you mean are httpsOnly, Always On, and a managed identity. Check the current template reference before you copy a year-old snippet into a real subscription. Bicep does not replace the expand/contract rule. An infrastructure change that renames the site is a new site, not a deploy.

What breaks

One pipeline that deploys the API, the function, and a full subscription template on every commit. A container tag latest moving under you after you thought production was frozen. A Bicep apply from a pull request build, so a fork or a feature branch edits production. Infrastructure apply belongs on main, behind the environment.

Say this

The API and the function are different artifacts and different identities. Images are deployed by digest. Infrastructure is a reviewed template, applied from main, and it cannot delete the estate because a developer pushed a branch.

Evening drill

1. Draw two artifacts from one build id.
2. Say why latest is not a release.
3. Name the pipeline permission you would refuse: contributor on the whole subscription.

Day 9 — Identity for pipelines

Outcome: you can draw the pipeline identity separately from the app identity, and you can explain OIDC so the workflow does not store a client secret.

Three identities, again, plus the pipeline

The user has a token. The API has a managed identity in the runtime SQL role. The migration has a different identity. The pipeline is a fourth. It must be allowed to deploy the app and, if it migrates, to apply DDL. Those can be two pipeline identities. Combining them means every build can change the schema. The deploy identity does not need to read customer tables.

Service connection and workload identity

In Azure DevOps, a service connection of type Azure Resource Manager is how the AzureWebApp task logs in. Workload identity federation uses a federated credential. There is no client secret to rotate in the variable group. The older service principal with a secret still works and still leaks when someone prints the variable or copies the connection. Prefer federation. Limit the connection to the resource group of the orders app, not the subscription, when the product allows that scope.

GitHub Actions and OIDC

GitHub requests a token for the job. Azure trusts that token when the subject matches the repository and the environment. The workflow asks for id-token: write. The login action exchanges the token. The federated credential subject includes the environment name when you bind it that way, so a pull request job that is not in the production environment cannot mint the production identity.

```
permissions:
  id-token: write
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
```

client-id is the application id, not a secret, though many teams still mask the tenant and subscription. The secret you eliminated is the client secret. Store nothing you can avoid. The federated credential's subject must match environment: production. A typo in either name fails closed, which is the failure you want.

What breaks

A client secret in a GitHub secret that every workflow in the repo can read. OIDC bound to the whole repository with no environment, so any branch workflow can deploy. Logging the Azure token to debug a 401. A service connection shared by every project in the organization, so a test app can deploy the orders API.

Say this

The pipeline has its own identity, federated, scoped to the resource group. Production deploy requires the production environment. There is no SQL password and no client secret in the YAML.

Evening drill

1. Name the four identities: user, API, migration, pipeline.
2. Say what the federated subject checks.
3. Explain why a pull request must not satisfy that subject.

Day 10 — Release safety

Outcome: you can refuse a release that cannot roll back, and you can write the ticket a person on call can follow.

The release ticket

The ticket is short. Build id, artifact, schema change, smoke command, rollback, and who is watching. If any line is empty, the release waits. A dashboard link belongs on the ticket. A novel belongs in the pull request.

```
Build: 2026.10.31.4
Artifact: orders-api from that build
Schema: add ExternalRef nvarchar(64) null
Smoke: GET /health/ready on the staging slot, then one authenticated GET /orders
Watch: p95 and SQL dependency for 20 minutes
Rollback: swap back to the previous slot
Blocked if: the migration drops or renames a column
```

Gates that are not theater

An approval from the person who wrote the change is a weak gate. An approval from the on-call owner, or a second team for schema, is a real one. An automated gate can query the health check after deploy and stop the swap. A manual approval with no checklist becomes a click. Write the checklist next to the environment. A lead removes a gate that everyone bypasses, or fixes the gate so it asks a question a human can answer.

When production is wrong after the swap

Look at the deploy marker and the chart before you read code. If the swap lines up and swap-back is safe, swap back. Then find the plan or the exception. This is the same sequence as the Azure day in the first roadmap. The new part is the pipeline evidence: the build id on the slot, the migration log, and the approval record. If you cannot find which bits are on the slot, the release process is not finished, even if today's incident ends.

What breaks

Hotfixing on the server and not committing it, so the next deploy restores the bug. Editing production app settings during the incident and forgetting they are not sticky, or that they are sticky and staging is now wrong. Declaring rollback impossible and then spending an hour on a forward fix while the error rate stays high.

Say this

The ticket names the artifact, the schema, the smoke check, and the rollback. If rollback is not real, the release is too big. After a bad swap I return to the previous bits first, then I diagnose.

Evening drill

1. Fill the release ticket for a nullable column and a slot swap.
2. Say the first action when p95 jumps at the deploy marker.
3. Name one gate you would delete because nobody can fail it honestly.

Day 11 — ASP.NET Core MVC

Outcome: you can separate model, view, and controller on an orders admin site, and you can say why this is not the Web API you designed in the first ten days.

The three parts

The model is the data and the rules the view needs. Often that is a view model, a type shaped for the screen, not the EF entity. The view is the Razor markup that turns that model into HTML. The controller accepts the HTTP request, calls the application, chooses the view, and chooses the status. The controller is thin. A controller that calculates tax, queries five tables, and builds HTML in strings has eaten the other two parts.

```
public sealed class OrderListViewModel
{
    public required IReadOnlyList<OrderRowViewModel> Orders { get; init; }
}

public sealed class OrdersController(IOrderQueries queries) : Controller
{
    [HttpGet("/admin/orders")]
    public async Task<IActionResult> Index(CancellationToken ct)
    {
        var rows = await queries.ListAsync(ct);
        return View(new OrderListViewModel { Orders = rows });
    }
}
```

return View(model) renders Views/Orders/Index.cshtml by convention. The view declares its model type and does not inject a DbContext. A link uses tag helpers so the URL stays tied to the action name. If you rename the action, the helper fails the build instead of failing in the browser.

```
@model OrderListViewModel
<table>
@foreach (var order in Model.Orders)
{
    <tr>
        <td>@order.Number</td>
        <td>
            <a asp-action="Detail" asp-route-id="@order.Id">Open</a>
        </td>
    </tr>
}
</table>
```

MVC, Razor Pages, and a Web API

MVC groups actions on a controller and views in a folder. Razor Pages group the page and its handler in one place, which fits simple forms. A Web API returns JSON and has no view. An admin site that humans click is a

fair MVC or Razor Pages app. The partner integration stays a Web API. Do not add views to the partner API so the admin site can share a controller. Share the application service. Keep two hosts, or two endpoint groups, with two auth styles: cookie for the browser, bearer token for the partner.

What breaks

Returning an EF entity to the view. The view then walks navigation properties and you have an N+1 during render, or you leak a concurrency token you did not mean to edit. Putting business rules in the cshtml because the if was easy there. The rule is untested and the API cannot reuse it.

Say this

The controller chooses the action and the view model. The view renders. The entity stays out of the view. Browser UI and partner API share the application service, not the controller.

Evening drill

1. Name model, view, and controller for the orders list.
2. Say why a view model is not an EF entity.
3. Say which auth the browser uses and which the partner uses.

Day 12 — MVC details that show up in review

Outcome: you can post a form safely, explain model binding, and name the filter that runs. You can reject a form that trusts a hidden admin flag.

A form, binding, and antiforgery

A GET shows the empty form. A POST binds form fields onto a view model and checks ModelState. The form includes an antiforgery token. ASP.NET Core checks it on the POST so another site cannot submit the form with the user's cookie. Tag helpers add the token when the form uses the form tag helper. A hand-built HTML form that omits it fails closed if ValidateAntiForgeryToken is on. That is the setting you want.

```
public sealed class CreateOrderViewModel
{
    [Required]
    public string Sku { get; set; } = "";

    [Range(1, 100)]
    public int Quantity { get; set; }
}

[HttpPost("/admin/orders")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(CreateOrderViewModel input, CancellationToken ct)
{
    if (!ModelState.IsValid)
        return View(input);
    var id = await handler.HandleAsync(input.ToCommand(), ct);
    return RedirectToAction(nameof(Detail), new { id });
}
```

```
<form asp-action="Create" method="post">
    <input asp-for="Sku" />
    <span asp-validation-for="Sku"></span>
    <input asp-for="Quantity" />
    <button type="submit">Create</button>
</form>
```

Redirect after a successful POST so a refresh does not submit again. If validation fails, return the same view with the model so the user sees the values and the messages. Do not put IsAdmin on the view model. A hidden field is still a field the browser can change. The user's role comes from the cookie claims. Over-posting is the same bug as on the API, with a friendlier UI.

Filters, areas, and TempData

An authorization filter runs before the action and can short-circuit with a challenge or a forbid result. An action filter runs around the action. A result filter runs around the view result. An exception filter is a local catch. Prefer the exception handler middleware for the site-wide error page so MVC and minimal endpoints

share it. An area is a folder of controllers for a subsection, such as Admin, with its own route prefix. TempData survives one redirect, which is how you show 'Order created' after RedirectToAction. It is not a shopping cart. Session is the wrong place for the order total if you have more than one server and no distributed session.

What breaks

A POST without antiforgery on a cookie-authenticated site. Binding an entity directly, so a posted RowVersion or TenantId overwrites the database. Validation only in jQuery, bypassed by a raw POST. An exception filter that swallows the error and returns the form as if it saved.

Say this

GET shows the form. POST validates, then redirects. Antiforgery protects the cookie. The view model lists the fields the user may send, and privilege is not one of them.

Evening drill

1. Walk a failed validation and a successful create, including the redirect.
2. Say what the antiforgery token is defending against.
3. Point at one field you would remove from a posted entity.

Day 13 — MVVM

Outcome: you can draw model, view, and view model, and you can explain why an ASP.NET Core MVC view model is not, by itself, the MVVM pattern.

The shape

MVVM is a client pattern. The view is the screen. The model is the domain data. The view model sits between them. The view binds to properties on the view model. The view model exposes commands for save and cancel. The view does not call the database. The view model does not construct buttons. This is the usual shape in WPF, .NET MAUI, WinUI, and some mobile XAML apps. The binding engine pushes property changes to the screen when the view model raises a change notification.

```
public sealed class CreateOrderViewModel : INotifyPropertyChanged
{
    private string _sku = "";
    private int _quantity = 1;

    public string Sku
    {
        get => _sku;
        set { _sku = value; OnPropertyChanged(); SaveCommand.RaiseCanExecuteChanged(); }
    }

    public int Quantity
    {
        get => _quantity;
        set { _quantity = value; OnPropertyChanged(); }
    }

    public RelayCommand SaveCommand { get; }

    public CreateOrderViewModel(OrderClient api)
    {
        SaveCommand = new RelayCommand(
            async () => await api.CreateAsync(Sku, Quantity),
            () => !string.IsNullOrWhiteSpace(Sku) && Quantity > 0);
    }

    public event PropertyChangedEventHandler? PropertyChanged;
    private void OnPropertyChanged([CallerMemberName] string? name = null)
        => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
}
```

The view binds Sku to a text box and SaveCommand to a button. The button disables itself when CanExecute is false. A unit test constructs the view model, sets Sku, and executes the command with a fake OrderClient. The test does not start the UI. That is the point of the pattern. INotifyPropertyChanged is the contract the binder listens to. Forgetting to raise it means the screen shows the old value after the code updates the field.

What MVC already has, and what it does not

An MVC type named OrderViewModel is a DTO for one request. The controller fills it, the view renders it, and the object dies at the end of the response. Nothing is observing property changes, because the HTML was already sent. Calling that MVVM confuses a panel. The honest sentence is: we use a view model so the view does not see the entity. The pattern of the server is still MVC. MVVM starts when a long-lived view binds to a long-lived object that raises change notifications and commands.

What breaks

Raising PropertyChanged for every keystroke that recalculates a huge order graph on the UI thread. The screen freezes. A view model that references a Window or a Button. You cannot test it, and you cannot reuse it on another client. Swallowing the API error inside the command so CanExecute stays true and the user thinks the order saved.

Say this

MVVM is a binding pattern for a live client. The view binds, the view model notifies and commands, the model is the domain. An MVC view model is a shape for one HTML response. I do not use the names as synonyms.

Evening drill

1. Draw view, view model, and model for a desktop create-order screen.
2. Say what a unit test can call without a window.
3. Say one sentence that keeps MVC and MVVM apart.

Day 14 — MVVM in front of ASP.NET Core

Outcome: you can place a MAUI or WPF client, a Blazor screen, and an Angular screen on the same API without forcing one pattern onto all three.

The API stays the API

POST /orders is the command. The client pattern does not change the idempotency key, the status codes, or the server-side price. The view model calls an OrderClient that uses HttpClient. The view model maps a 400 onto error properties the view already binds. It does not parse HTML. The API does not return a view model for WPF.

```
public sealed class OrderClient(HttpClient http)
{
    public async Task<OrderDto> CreateAsync(string sku, int quantity, string key, CancellationToken
ct)
    {
        var request = new HttpRequestMessage(HttpMethod.Post, "orders")
        {
            Content = JsonContent.Create(new { sku, quantity })
        };
        request.Headers.Add("Idempotency-Key", key);
        var response = await http.SendAsync(request, ct);
        response.EnsureSuccessStatusCode();
        return (await response.Content.ReadFromJsonAsync<OrderDto>(ct))!;
    }
}
```

Blazor and Angular are cousins, not MVVM by slogan

A Blazor component has a view, the razor file, and state in the component class. If that state is a separate class with methods the markup calls, you are close to a view model. Blazor does not require INotifyPropertyChanged. It re-renders after an event or after you call StateHasChanged. An Angular component plus a service, from the Angular guide, is the same idea: the template binds, the class holds state, the service calls HTTP. Use those names with the team you are on. In a design review, say what is bound, what calls the API, and what you can test without the UI. That is the MVVM benefit even when the framework spells it differently.

Where the pattern earns its cost

A one-page internal tool does not need a relay command hierarchy. A desktop app with ten screens on one API does, or every screen will new up HttpClient and copy the error handling. Share the client and the auth handler. Keep each view model on one screen. A view model that is also a cache for the whole company catalog has become a service. Move the cache.

What breaks

Each view model creating its own HttpClient. Sockets run out, the same way they do on the server. Catching every exception and setting Error = ex.ToString(), which shows a stack to the user and may include a server path. Treating a 409 concurrency conflict as a generic failure, so the user retries and overwrites someone else.

Say this

The API contract stays MVC or minimal API on the server. The client holds a view model or a component and calls HttpClient through one typed client. I test the view model without the screen. I do not pretend Blazor, Angular, and WPF are the same pattern. I do say they share the split.

Evening drill

1. Place browser MVC, a Web API, and a MAUI view model in one diagram.
2. Say which piece owns the idempotency key on a retry.
3. Name the test that does not start a device.

Day 15 — ASP.NET Core, the parts after the first roadmap

Outcome: you can go past the pipeline sketch. You can place middleware, filters, and endpoint filters, and you can add caching and a background worker without capturing a DbContext.

Middleware, MVC filters, and endpoint filters

Middleware sees the HTTP request and runs for every endpoint that is downstream of it. MVC filters see an MVC action. Endpoint filters see a specific minimal API or a group. Use middleware for the correlation id, the exception page, and authentication. Use an MVC filter for an admin convention, such as a filter that adds a cache header on one controller. Use an endpoint filter for a validation check that belongs to one route group. A business rule in middleware is invisible to a queue consumer that calls the same handler later.

```
var orders = app.MapGroup("/orders").RequireAuthorization("orders.write");
orders.MapPost("/", async (CreateOrderRequest body, CreateOrderHandler handler, CancellationToken
ct) =>
{
    var result = await handler.HandleAsync(body.ToCommand(), ct);
    return result.Ok
        ? Results.Created($"/orders/{result.Value!.Id}", result.Value)
        : Results.ValidationProblem(new Dictionary<string, string[]> { ["body"] = [result.Error!]
});
});
```

Output cache and rate limits

A reference list that changes hourly can be output-cached. An order the user just posted must not be cached for another user. Vary the cache by the tenant claim, or do not cache it. A public cache of a tenant-specific body is a data leak. Rate limiting protects the process from one client. It is not the only capacity plan. The SQL pool still has a limit. Return 429 with Retry-After. Do not rate-limit health checks or the site will mark itself unhealthy and restart.

```
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("orders", o =>
    {
        o.Window = TimeSpan.FromSeconds(1);
        o.PermitLimit = 20;
        o.QueueLimit = 0;
    });
});
app.UseRateLimiter();
app.MapGet("/catalog/categories", ListCategories).RequireRateLimiting("orders");
```

The numbers are an example policy, not a platform quota. Set them from the real client and measure. OutputCache on a GET of categories is reasonable. OutputCache on POST /orders is a defect.

A background worker that owns a scope

IHostedService or BackgroundService is a singleton. It must not take DbContext in the constructor. Create a scope per pass, do one unit of work, dispose the scope. Pass the stopping token into the delay and into EF. Log the failure and continue the loop unless the process should stop. A worker that throws on the first SQL timeout takes the whole site down if the host is configured to stop when a background service faults.

```
protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    while (!stoppingToken.IsCancellationRequested)
    {
        await using var scope = scopeFactory.CreateAsyncScope();
        var relay = scope.ServiceProvider.GetRequiredService<OutboxRelay>();
        await relay.PublishBatchAsync(stoppingToken);
        await Task.Delay(TimeSpan.FromSeconds(2), stoppingToken);
    }
}
```

What breaks

Caching a personalized response in a shared cache. A rate limit so low that a single office NAT looks like an attack and locks out a tenant. A hosted service that uses the root service provider for the life of the process and keeps one DbContext open. Middleware that reads the body and does not rewind it, so the action sees an empty body.

Say this

Middleware is for every request. Filters are for a route or an MVC action. I cache public reference data, varied by tenant if the body is private. A background worker opens a scope per batch. It does not capture DbContext.

Evening drill

1. Place correlation id, auth, and a single-route validator in middleware or filters.
2. Say which GET you would cache and which you would not.
3. Explain the scope inside the hosted service.

Day 16 — ASP.NET Core in production and in tests

Outcome: you can show logging, health, versioning, and one integration test that boots the real pipeline without a production database.

Logs, health, and versioning

Log the correlation id, the route, the status, and the elapsed time. Do not log the body when it can contain a name, an address, or a card. Health has two endpoints. Liveness is the process. Readiness includes a cheap SQL check. The App Service health probe uses readiness. API versioning for an enterprise partner is a URL segment, /v1/orders, until you have a reason to hide it in a header. Run v1 and v2 together. Measure calls on v1 before you delete it. The pipeline deploys both. A version is a contract, not a folder you rename on Friday.

```
builder.Services.AddHealthChecks()
    .AddDbContextCheck<AppDbContext>("sql", tags: ["ready"]);

app.MapHealthChecks("/health/live", new HealthCheckOptions { Predicate = _ => false });
app.MapHealthChecks("/health/ready", new HealthCheckOptions
{
    Predicate = check => check.Tags.Contains("ready")
});
```

WebApplicationFactory

An integration test constructs the host with WebApplicationFactory, replaces the database or the auth handler, and sends HTTP through the test client. The test proves routing, auth, and model validation together. It does not replace a SQL test of the query. Use it for the contract: an anonymous POST is 401, a body without a SKU is 400, a valid body with a test double for the handler returns 201. Dispose the factory. Do not point it at the production connection string in appsettings.

```
public class OrdersApiTests : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly WebApplicationFactory<Program> _factory;
    public OrdersApiTests(WebApplicationFactory<Program> factory) => _factory = factory;

    [Fact]
    public async Task Post_without_a_token_is_unauthorized()
    {
        var client = _factory.CreateClient();
        var response = await client.PostAsJsonAsync("/orders", new { sku = "AB-1", quantity = 1 });
        Assert.Equal(HttpStatusCode.Unauthorized, response.StatusCode);
    }
}
```

Program must be visible to the test project. The usual approach is a public partial Program in the API project. The test replaces authentication with a handler that always fails, or always succeeds with a tenant claim,

depending on the case. Two tests are more honest than one test that disables auth for the whole class and then claims to have tested auth.

Configuration and options, at release time

appsettings.json is the default. The environment name selects appsettings.Production.json. Environment variables override both. Azure App Service settings become environment variables. Key Vault references land in configuration before the host finishes starting, when the identity can read them. ValidateOnStart throws if Pricing:Rate is missing, and the slot fails before swap if the health check starts the process. A secret in appsettings.Production.json that is copied to the repo is still a secret in Git, whatever the file name says.

What breaks

A liveness probe that calls SQL, so a brief database blip restarts every instance. A factory test that uses the real startup connection and deletes rows. Versioning by copying the controller and forgetting the authorization policy on v2. Logging the request body in Development and leaving that switch on in Production.

Say this

Readiness checks SQL. Liveness does not. Versions overlap. WebApplicationFactory proves the HTTP contract, including 401. Secrets come from the environment or Key Vault, not from a json file in Git.

Evening drill

1. Write the two health URLs and what each may touch.
2. Name the assertion of the anonymous POST test.
3. Say the order of configuration overrides in one sentence.

Day 17 — Microservices past the first sketch

Outcome: you can go beyond the outbox drawing from the first roadmap. You can say who owns the service, how it deploys, and how a client discovers a contract without sharing the EF project.

A boundary you can staff

A service is a deployable with its own data, its own pipeline, and a team that can be on call for it. A project folder is a module. If orders and catalog still ship in one pull request every week, they are one team and should stay one deployable until that stops being true. The extra pieces you accept on the day you split are: a contract, a dashboard, an on-call rotation, and a way to roll the service back without rolling the other one back.

Contracts without a shared database model

Publish an OpenAPI document or a small message schema. The consumer writes a test against that contract. The provider's CI runs the contract. A shared Orders.Domain project referenced by the email service and the API is a distributed monolith: one entity change forces two deploys. The message OrderCreated carries an id, a version, a correlation id, and the fields the email needs. It does not carry the EF entity.

```
public sealed record OrderCreated(
    Guid MessageId,
    string CorrelationId,
    int OrderId,
    string Number,
    DateTimeOffset OccurredUtc);
```

A gateway in front, not a shared database behind

YARP, or the platform gateway, routes /orders to the orders service and /catalog to the catalog service. The browser still sees one host. Each service still has its own database and its own login. The gateway authenticates once if that is the product choice, and forwards the tenant claim. It does not open SQL. A gateway that also contains tax rules becomes the new monolith. Keep rules in the service that owns the data.

```
{
  "ReverseProxy": {
    "Routes": {
      "orders": {
        "ClusterId": "orders",
        "Match": { "Path": "/orders/{**catch-all}" }
      }
    },
    "Clusters": {
      "orders": {
        "Destinations": {
          "a": { "Address": "https://app-orders.internal" }
```

```
        }
      }
    }
  }
}
```

The address is an example. Production uses the name your network actually has, and the gateway's identity is allowed to call the service. Internal services still authenticate the forwarded user or the gateway identity. A network rule alone is not authorization if the network is wide.

What breaks

Splitting the deploy and keeping one database with cross-schema joins. Two pipelines that must run in a fixed order every release. A shared NuGet of entities that versions in lockstep. A gateway timeout longer than the service timeout, so the user waits on a dead call the service already abandoned.

Say this

I split when a team can own the deploy, the data, and the on-call. The contract is a message or OpenAPI, not the EF model. The gateway routes. It does not own the business rule or the database.

Evening drill

1. Name the four things you add on the day you split catalog from orders.
2. Say what the email service is allowed to reference.
3. Explain why the gateway is not the place for tax.

Day 18 — One service, one pipeline, one failure

Outcome: you can describe a single orders service as it would exist after the split, including the pipeline path filter, the health check, and the user-visible failure.

The slice

The orders service is an ASP.NET Core API on App Service. Its pipeline builds on every change under src/Orders and deploys only that artifact. Catalog has a path filter of its own. A change to email does not recycle orders. The database is orders, with the runtime role and the migration identity from the first roadmap. The outbox and the idempotency table stay in that database. Inventory is not a table in this database. It is a call or a message.

```
trigger:
  branches:
    include: [main]
  paths:
    include:
      - src/Orders/**
      - pipelines/orders.yml

pr:
  branches:
    include: [main]
  paths:
    include:
      - src/Orders/**
```

Failure the product can see

If inventory is down and the business said the customer may still place an order, the API writes Pending and the outbox. The user sees 201 and a status. If tax is required to price the order, the API returns 503 after a short timeout and does not write a wrong price. The pipeline did not cause that choice, but the dashboard the pipeline deploys with must show outbox age and the 503 rate. A service without those two numbers is not ready to be on its own pager.

Rollback of one service

Swap the orders slot back. Catalog stays. That works when the message contract was compatible. Adding an optional field is compatible. Changing the meaning of Number is not, and then a rollback of orders with a new catalog still in production is a lie. Contract changes use the same expand habit as database columns: add, deploy consumers that tolerate both, then remove. The pipeline does not deploy two services as one atomic release unless you have a reason you can say out loud.

What breaks

A path filter that forgets the shared library, so orders keeps running an old copy of the money type until someone notices. A health check that always returns 200 while the outbox is an hour old. Rolling back orders when the new event field was required and catalog already depends on it.

Say this

One service, one artifact, one path filter, one database, one slot rollback. The user either gets a 201 with a pending status or a short 503. I do not deploy the estate to change a line in orders.

Evening drill

1. Write the path filter for orders and say what it must also include.
2. Choose 201 or 503 for a down tax service and justify it.
3. Say when a single-service rollback is unsafe.

Day 19 — AIPDLC, the AI-assisted lifecycle

Outcome: you can place Cursor and Copilot inside the lifecycle without handing them the release. AIPDLC here means the AI-assisted product development life cycle: the same discover, design, build, test, release, and operate loop, with a model allowed to draft and a human required to sign.

Where a draft is useful

Stage	The model may draft	A human still signs
Discover	A first cut of stories and questions	The problem, the metric, and what is out of scope
Design	An ADR outline, a sequence, threats to consider	The boundary, the data owner, and the rollback
Build	A method, a test, a YAML step, a Razor field	The diff, the auth, the SQL, the migration
Review	A list of risks to look at	The approval, especially on schema and identity
Test	Extra cases around the ones you named	The assertion that protects money or a tenant
Release	A release-ticket draft from the diff	The approval and the swap
Operate	A summary of logs you paste, with secrets removed	Rollback versus fix-forward

The model is a fast colleague who has not been on call for your system. It will invent a reasonable Azure setting. You check the current document or you do not ship the number. It will satisfy the test you asked for and miss the tenant test you forgot to mention. Your prompt is a specification. A vague prompt produces a vague edit across fifty files.

What you do not put in the prompt

Production connection strings, tokens, customer rows, and private keys do not go into chat. A stack trace is useful after you remove the connection string it printed. A failing test and the method under test are the right context. An entire repo paste is not. If the tool offers a privacy mode or a business agreement that keeps prompts out of training, the company decides that. You still do not paste a secret, because the secret is now in a log you do not control.

A worked pass on one story

Story: a partner retry must not create a second order. You write the acceptance test yourself, or you ask the tool to draft it and you fix the assertion until a missing unique index would fail it. You ask for the smallest change: a key column, a unique index, a check in the handler, and the migration as expand. You read the migration SQL. You ask for the pipeline impact only if the project file changed. You do not ask the tool to also split a microservice in the same session. The pull request stays reviewable. The tool does not approve it.

```
Prompt shape that stays small:

Context: src/Orders.Api/CreateOrderHandler.cs and the test project.
Task: store an Idempotency-Key with a hash of the body in the same transaction as the order.
```

```
Do: unique (TenantId, Key). Same key and same body returns the first result.
Same key and a different body returns 409.
Do not: add a new service, change auth, or edit the pipeline.
Tests: one retry test must fail if the second insert happens.
```

What breaks

Accepting a green diff you did not read. The tool added a catch that returns 200. A generated test that asserts true. A prompt that includes a customer export so the model can 'use realistic data'. A release pipeline edited by an agent and merged because the build passed. Generated code is still your code when it ships.

Say this

AI drafts inside the lifecycle. I sign the boundary, the test that protects money, the migration, and the release. I keep the prompt small, I keep secrets out, and I read the diff.

Evening drill

1. For each lifecycle stage, say one thing you would let a model draft.
2. Rewrite the idempotency prompt so it forbids a drive-by refactor.
3. Name three things that never go into the chat box.

Day 20 — Cursor and GitHub Copilot

Outcome: you can use both tools as a senior uses a junior: clear tasks, a diff you read, and a refusal when the change is too wide. The habits overlap. The files they read for project rules do not.

What each tool is for

GitHub Copilot sits in the editor and in GitHub. Inline suggestions complete the line or the method you are already typing. Chat answers a question about the selection. A coding agent can open a pull request. You review that pull request with the same CODEOWNERS rules as a human colleague. Cursor is the editor you can ask to change several files, run a focused command, and show a diff. Agent mode edits. Plan mode is for a change you want designed before files move. Ask, or a read-only pass, is how you learn a pipeline file without rewriting it. Use plan when the task touches the pipeline, auth, or the data model. Use a direct edit when the task is one method and one test.

Context is the trick

Name the files. In Cursor, attach the handler and the test. In Copilot, point the chat at the selection or the file. A question with no file becomes a generic lecture. A question with the failing test and the method becomes a patch. Close the mental scope the same way: one bug, one acceptance check. If the answer wanders into a new architecture, stop and restate the boundary.

```
Cursor, agent, after you have the test:

Open CreateOrderHandler and IdempotencyTests.
Make the retry test pass.
Keep the unique key in the same SaveChanges as the order.
Do not change the YAML or the auth handler.
Show me the migration SQL before you add more files.

Copilot, in the test file, with the method selected:

Write a test where the same Idempotency-Key and a different body returns 409.
Use the existing test host. Do not add a new framework.
```

Project rules the tools can see

Copilot reads .github/copilot-instructions.md for the repository, and path-specific files under .github/instructions when the file you are editing matches. Cursor rules live with the project and tell the agent how this repo builds, tests, and reviews. Write the rules you already believe: cancellation tokens go to EF, the client does not send the price, migrations are expand-only, do not commit secrets, do not edit the pipeline unless asked. A rule you do not follow yourself trains the tool to be ignored. Keep the file short. A 40-page rule file is a novel. The tool will not hold it all.

```
# .github/copilot-instructions.md
This is an ASP.NET Core orders API on .NET 10.
```

```
Pass CancellationToken into EF and HttpClient.
Do not bind price, tenant, or roles from the request body.
Schema changes are expand then contract.
Do not add a new project or a new package unless the prompt asks.
Tests use the existing test project. Prefer one behavior per test.
```

Tricks that save a review

- Ask for the test first, watch it fail, then ask for the implementation. A test written after a green implementation often encodes the bug.
- Ask the tool to explain a YAML stage or a Razor form before you ask it to rewrite one. Understanding is cheaper than reverting.
- When the edit is wrong, say what is wrong in one sentence and point at the line. Do not start a new chat that has forgotten the constraint.
- Reject a suggestion that catches Exception and returns success, that logs a token, or that uses a SQL password.
- For a pipeline change, read the trigger and the environment before you run it. A suggested on: push with a deploy job is a production change.
- For MVVM, ask for a view model test with a fake client. Reject a suggestion that new's a window.
- For MVC, ask the form to include antiforgery and a view model. Reject a posted EF entity.
- Let formatters fix braces. Do not spend a session on style the editor already owns.
- If you cannot explain the diff, do not commit it. That rule is older than these tools and still decides the interview.

A week of practice after the 20 days

Repeat one pipeline file until you can mark trigger, test, artifact, environment, and rollback without notes. Repeat one MVC form and one view model until the difference is boring. Then take a real story from your backlog and run it through the AIPDLC table: what the tool drafted, what you changed, and what you refused. That story is more useful in an interview than a list of product names.

What breaks

Trusting an inline completion in a migration or an auth handler because it looked fluent. Giving an agent permission to push to main. Pasting a production log with a connection string into chat to get a quicker diagnosis. A rule file that says 'be careful' and never mentions tenant, price, or secrets. The tool cannot invent your invariants if you hide them.

Say this

I point the tool at the file and the test. I keep the task small. Copilot instructions and Cursor rules state the invariants. I read the diff, I refuse secrets and drive-by architecture, and I still own the release.

Evening drill

1. Write five lines of copilot-instructions for the orders API.
2. Rewrite a vague prompt, 'make orders better', into a bounded one.
3. Say which tool mode you want before a schema change, and why.

The map back to the first 10 days

Days 1 to 10 of the first roadmap are still the language, the API, SQL, Azure hosts, and the lead conversation. These 20 days are how that system is built with other people and with these tools. If a panel asks only one question from this set, answer with a pull request, a pipeline environment, a rollback, and the line you would not let a model merge. That is the same lead signal as the first guide: a decision, a cost, and a way back.